

CASE STUDY:

Name:Shyam sundhar. Y

Reg.no:192424082

Code:CSA0603

Course:Design and analysis of algorithm

Q34. Case Study: Robotics Vision System (Matrix Multiplication)

A robotics system processes visual data using matrix transformations. Analyze how divide and conquer matrix multiplication enhances performance. Identify computational challenges in real-time processing. Apply optimized techniques such as Strassen's method. Analyze complexity and justify the solution.

Title:

Case Study: Robotics Vision System Using Matrix Multiplication (Divide and Conquer with Strassen's Algorithm)

Description of the Real-World Problem:

Modern robotic systems use cameras to understand their surroundings. Every image captured by a robot is represented as a matrix of pixel values. Tasks such as object detection, obstacle avoidance, image rotation, feature extraction, and 3D transformations require frequent matrix multiplication.

In autonomous robots, drones, self-driving vehicles, and industrial robots, these matrix operations must be completed within milliseconds. Traditional matrix multiplication becomes computationally expensive as image resolution increases. Therefore, faster algorithms such as Divide and Conquer Matrix Multiplication (Strassen's Algorithm) are used to improve processing speed.

Problem Statement:

Develop an efficient matrix multiplication technique for a robotics vision system using the Divide and Conquer approach (Strassen's Algorithm). Analyze how the algorithm improves computational performance compared to the conventional matrix multiplication method. Evaluate its complexity, advantages, limitations, and suitability for real-time robotic vision applications.

Algorithm Used:

Strassen's Matrix Multiplication Algorithm (Divide and Conquer)

Steps

1. Input two square matrices A and B.
2. If matrix size is 1×1 :

- Multiply directly.
3. Divide each matrix into four equal submatrices.
 4. Compute seven intermediate matrices:
 - $M1 = (A11 + A22)(B11 + B22)$
 - $M2 = (A21 + A22)B11$
 - $M3 = A11(B12 - B22)$
 - $M4 = A22(B21 - B11)$
 - $M5 = (A11 + A12)B22$
 - $M6 = (A21 - A11)(B11 + B12)$
 - $M7 = (A12 - A22)(B21 + B22)$
 5. Calculate the result submatrices:
 - $C11 = M1 + M4 - M5 + M7$
 - $C12 = M3 + M5$
 - $C21 = M2 + M4$
 - $C22 = M1 - M2 + M3 + M6$
 6. Combine the four submatrices into the final result matrix.
 7. Display the multiplied matrix.

Justification for Choosing the Algorithm (Why This Algorithm?):

Strassen's algorithm is selected because:

- It reduces the number of multiplications from **8 to 7** at each recursion level.
- Matrix multiplication is the most time-consuming operation in robotics vision.
- Faster multiplication enables real-time image processing.
- Efficient for high-resolution images.
- Widely used in scientific computing and computer vision.
- Suitable for parallel processing architectures.

Complexity Analysis:

Operation	Time Complexity
Conventional Matrix Multiplication	$O(n^3)$

Operation	Time Complexity
Divide & Conquer (Standard)	$O(n^3)$
Strassen's Algorithm	$O(n^{2.81})$
Space Complexity	$O(n^2)$

Analysis:

- Conventional multiplication performs **n^3 multiplications**.
- Strassen reduces multiplication operations significantly.
- As matrix size increases, execution time decreases compared to the traditional approach.

Manual Execution (Step-by-Step Process):

Let

Matrix A

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Matrix B

$$\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Step 1

Divide matrices into four submatrices.

$$A_{11} = 1$$

$$A_{12} = 2$$

$$A_{21} = 3$$

$$A_{22} = 4$$

$$B_{11} = 5$$

$$B_{12} = 6$$

$$B_{21} = 7$$

$$B_{22} = 8$$

Step 2

Calculate M1 to M7

$$M1 = (1+4)(5+8) = 65$$

$$M2 = (3+4)5 = 35$$

$$M3 = 1(6-8) = -2$$

$$M4 = 4(7-5) = 8$$

$$M5 = (1+2)8 = 24$$

$$M6 = (3-1)(5+6) = 22$$

$$M7 = (2-4)(7+8) = -30$$

Step 3

Compute Result

$$C_{11}$$

$$= 65 + 8 - 24 - 30$$

$$= 19$$

$$C_{12}$$

$$= -2 + 24$$

$$= 22$$

$$C_{21}$$

$$= 35 + 8$$

$$= 43$$

$$C_{22}$$

$$= 65 - 35 - 2 + 22$$

$$= 50$$

Final Matrix

$$\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Applications:

- Robotics Vision Systems
- Object Detection
- Face Recognition
- Image Transformation
- Medical Image Processing
- Autonomous Vehicles
- Drone Navigation
- Satellite Image Analysis
- Computer Graphics
- Artificial Intelligence

Advantages:

- Faster than conventional matrix multiplication.
- Reduces multiplication operations.
- Suitable for large matrices.
- Supports divide-and-conquer paradigm.
- Improves robotics image processing speed.
- Efficient for parallel execution.
- Better performance in scientific computing.

Limitations:

- More addition and subtraction operations.
- Recursive implementation is complex.
- Extra memory is required.
- Less efficient for very small matrices.
- Numerical errors may accumulate in floating-point operations.

Comparison with Other Algorithms:

Algorithm	Time Complexity	Multiplications	Suitable For
Conventional Matrix Multiplication	$O(n^3)$	n^3	Small matrices
Divide & Conquer Matrix Multiplication	$O(n^3)$	8 recursive multiplications	Medium matrices
Strassen's Algorithm	$O(n^{2.81})$	7 recursive multiplications	Large matrices, Robotics Vision
Coppersmith-Winograd	Better theoretical complexity	Very Complex	Research applications

Program:

```
M4 = strassen(A22, subtract_matrices(B21, B11))
M5 = strassen(add_matrices(A11, A12), B22)
M6 = strassen(subtract_matrices(A21, A11), add_matrices(B11, B12))
M7 = strassen(subtract_matrices(A12, A22), add_matrices(B21, B22))
C11 = add_matrices(subtract_matrices(add_matrices(M1, M4), M5), M7)
C12 = add_matrices(M3, M5)
C21 = add_matrices(M2, M4)
C22 = add_matrices(subtract_matrices(add_matrices(M1, M3), M2), M6)
new_matrix = []
for i in range(n // 2):
    new_matrix.append(C11[i] + C12[i])
for i in range(n // 2):
    new_matrix.append(C21[i] + C22[i])
return new_matrix

A = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
]
B = [
    [16, 15, 14, 13],

```

```
1 def add_matrices(A, B):
2     n = len(A)
3     return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]
4 def subtract_matrices(A, B):
5     n = len(A)
6     return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]
7 def split_matrix(M):
8     n = len(M)
9     mid = n // 2
10    A11 = [row[:mid] for row in M[:mid]]
11    A12 = [row[mid:] for row in M[:mid]]
12    A21 = [row[:mid] for row in M[mid:]]
13    A22 = [row[mid:] for row in M[mid:]]
14    return A11, A12, A21, A22
15 def strassen(A, B):
16     n = len(A)
17     if n == 1:
18         return [[A[0][0] * B[0][0]]]
19     A11, A12, A21, A22 = split_matrix(A)
20     B11, B12, B21, B22 = split_matrix(B)
21     M1 = strassen(add_matrices(A11, A22), add_matrices(B11, B22))
22     M2 = strassen(add_matrices(A21, A22), B11)
23     M3 = strassen(A11, subtract_matrices(B12, B22))
```

```
[12, 11, 10, 9],  
[8, 7, 6, 5],  
[4, 3, 2, 1]  
]  
result = strassen(A, B)  
for row in result:  
    print(row)
```

Output

```
[80, 70, 60, 50]  
[240, 214, 188, 162]  
[400, 358, 316, 274]  
[560, 502, 444, 386]
```

Conclusion:

Robotics vision systems rely heavily on matrix multiplication for image transformations and object recognition. Conventional matrix multiplication becomes inefficient for large image matrices due to its cubic time complexity. Strassen's Algorithm applies the divide-and-conquer strategy to reduce the number of multiplication operations, achieving a time complexity of $O(n^{2.81})$. This improvement enables faster image processing, making it suitable for real-time robotic applications such as autonomous navigation, industrial automation, medical imaging, and computer vision. Although it introduces additional memory usage and recursive overhead, Strassen's Algorithm provides significant performance benefits for large-scale matrix operations and is an effective choice for robotics vision systems.